SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I R. Indhu (292524332) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform — Normalize to 3NF

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Assume one order can contain multiple products.

Functional Dependencies

We can identify:

- OrderID → CustID
- CustID → CustName
- ProdID → ProdName, Price
- (OrderID, ProdID) → Qty

Since an order can contain several products, the combination of OrderID and ProdID identifies an order item.

Candidate Key

(OrderID, ProdID)

Because:

- OrderID identifies the customer.
- ProdID identifies the product.
- OrderID + ProdID identifies the quantity ordered.

Partial Dependencies

We have:

OrderID → CustID

and:

ProdID → ProdName, Price

Both OrderID and ProdID are parts of the composite key.

Therefore, partial dependencies exist.

So the relation is **not in 2NF**.

Decomposition

ORDER

ORDER(OrderID, CustID)

Primary Key: **OrderID**

CUSTOMER

CUSTOMER(CustID, CustName)

Primary Key: **CustID**

PRODUCT

PRODUCT(ProdID, ProdName, Price)

Primary Key: **ProdID**

ORDER_ITEM

ORDER_ITEM(OrderID, ProdID, Qty)

Primary Key:

(OrderID, ProdID)

Transitive Dependency

OrderID → CustID

and

CustID → CustName

Therefore:

OrderID → CustName

This transitive dependency is removed by creating the CUSTOMER table.

Final 3NF Design

Relation	Attributes	Key
CUSTOMER	CustID, CustName	CustID
ORDER	OrderID, CustID	OrderID
PRODUCT	ProdID, ProdName, Price	ProdID
ORDER_ITEM	OrderID, ProdID, Qty	OrderID + ProdID

Conclusion: The original order relation is decomposed into 3NF by removing partial and transitive dependencies.

Q2. Hospital — Normalize to BCNF

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Functional Dependencies

Assume:

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Candidate Key

PID

PID determines all other attributes.

Therefore:

PID is the candidate key.

Transitive Dependencies

We have:

PID → DocID

and:

DocID → DocName, Specialization

Therefore:

PID → DocName, Specialization

Similarly:

PID → WardNo

and:

WardNo → WardName

Therefore:

PID → WardName

These are transitive dependencies.

BCNF Analysis

For BCNF, the determinant of every non-trivial FD must be a superkey.

But:

DocID → DocName, Specialization

DocID is not a superkey of the original relation.

Also:

WardNo → WardName

WardNo is not a superkey.

Therefore, the original relation violates BCNF.

Decomposition

PATIENT

PATIENT(PID, PName, DocID, WardNo)

Primary Key: PID

DOCTOR

DOCTOR(DocID, DocName, Specialization)

Primary Key: DocID

WARD

WARD(WardNo, WardName)

Primary Key: WardNo

Conclusion

The decomposition removes the transitive dependencies and ensures that every determinant is a candidate key in its respective relation.

Therefore, the design satisfies **BCNF**.

Q3. University Enrollment — 2NF

Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Functional Dependencies

- SID → SName
- CourseID → CourseName, Faculty
- (SID, CourseID) → Grade

Candidate Key

(SID, CourseID)

Why?

SID identifies the student, while CourseID identifies the course.

Together they identify one enrollment record and its grade.

Partial Dependencies

We have:

SID → SName

Here SName depends only on part of the composite key.

Also:

CourseID → CourseName, Faculty

These attributes depend only on CourseID.

Therefore, the relation is **not in 2NF**.

Decomposition

STUDENT

STUDENT(SID, SName)

Primary Key: SID

COURSE

COURSE(CourseID, CourseName, Faculty)

Primary Key: CourseID

ENROLLMENT

ENROLLMENT(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

Conclusion

The partial dependencies are removed.

Therefore, the resulting relations are in **2NF**.

Q4. Airline Booking — Normalize up to BCNF

Given Relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assume:

- A passenger can book flights.
- A flight has a departure city and arrival city.
- A flight operates on a particular date.
- A passenger gets a seat for a particular flight/date.

Functional Dependencies

Possible dependencies:

- PassID → PassName
- FlightNo → DeptCity, ArrCity
- (FlightNo, Date, PassID) → Seat

The exact candidate key depends on the business rules. Under the assumption that a passenger can book a flight only once for a particular date:

Candidate Key = (FlightNo, Date, PassID)

Partial Dependencies

PassID → PassName

and:

FlightNo → DeptCity, ArrCity

These depend on parts of the composite key.

Therefore, decomposition is required.

Decomposition

PASSENGER

PASSENGER(PassID, PassName)

Primary Key: PassID

FLIGHT

FLIGHT(FlightNo, DeptCity, ArrCity)

Primary Key: FlightNo

BOOKING

BOOKING(FlightNo, Date, PassID, Seat)

Primary Key:

(FlightNo, Date, PassID)

BCNF Check

In PASSENGER:

PassID → PassName

PassID is a key.

In FLIGHT:

FlightNo → DeptCity, ArrCity

FlightNo is a key.

In BOOKING:

(FlightNo, Date, PassID) → Seat

The determinant is the key.

Therefore, under the stated assumptions, the decomposed relations satisfy BCNF.

Conclusion: The airline booking relation is normalized up to BCNF.

Q5. HR System — Normalize to 3NF

For this question, you need to **state reasonable assumptions**, because the question does not provide an exact schema.

Use:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, JobID, JobTitle)

Functional Dependencies

- EmpID → EmpName, DeptID, ManagerID, JobID

- DeptID → DeptName, ManagerID
- ManagerID → ManagerName
- JobID → JobTitle

Candidate Key

EmpID

Transitive Dependencies

We have:

EmpID → DeptID

and:

DeptID → DeptName

Therefore:

EmpID → DeptName

Similarly:

EmpID → ManagerID

and:

ManagerID → ManagerName

Therefore:

EmpID → ManagerName

Also:

EmpID → JobID

and:

JobID → JobTitle

Therefore:

EmpID → JobTitle

These are transitive dependencies.

Decomposition

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, JobID)

DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

MANAGER

MANAGER(ManagerID, ManagerName)

JOB

JOB(JobID, JobTitle)

Conclusion

The transitive dependencies are removed by separating department, manager, and job information.

The resulting design satisfies **3NF**.

Q6. Library — MVD and 4NF

Given Relation

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

For this case, we need to identify independent multi-valued information.

Assume a member can borrow multiple books and a book may have multiple independent authors.

Possible MVD:

MemberID →→ BookID

If the member's book borrowing information is independent of another multi-valued attribute, an MVD exists.

Example

A member may borrow:

- Book B1
- Book B2

If another independent set of information exists, storing both in the same relation can produce unnecessary combinations.

4NF Rule

For every non-trivial MVD:

X →→ Y

X must be a superkey.

If MemberID is not a superkey, the relation violates 4NF.

Decomposition

A suitable decomposition can separate independent facts, for example:

MEMBER

MEMBER(MemberID, MemberName)

BOOK

BOOK(BookID, Title, Author, Genre)

BORROW

BORROW(MemberID, BookID, BorrowDate)

Conclusion

Independent multi-valued facts are separated into different relations, reducing redundancy and satisfying the requirements of **4NF**, provided the assumed MVDs hold.

Important: For this question, write your assumptions clearly because the original question does not specify the exact MVDs.

Q7. Telecom Billing — Keys and Functional Dependencies

Since the question does not give an exact schema, use a reasonable telecom billing schema.

Relation

BILLING(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Amount)

Functional Dependencies

- CustomerID → CustomerName
- CustomerID → PhoneNo
- PlanID → PlanName
- CallID → CustomerID, CallDate, Duration, Amount
- CustomerID → PlanID

Candidate Keys

If CallID uniquely identifies each call:

Candidate Key = CallID

Therefore:

Primary Key = CallID

Other attributes such as CustomerID and PlanID are not candidate keys because they cannot identify every call.

Transitive Dependency

We have:

CallID → CustomerID

and:

CustomerID → CustomerName, PhoneNo, PlanID

Therefore:

CallID → CustomerName, PhoneNo, PlanID

Also:

PlanID → PlanName

Therefore:

CallID → PlanName

These are transitive dependencies.

Summary

Attribute	Determines
CustomerID	CustomerName, PhoneNo, PlanID
PlanID	PlanName
CallID	CustomerID, CallDate, Duration, Amount

Conclusion

Before normalization, candidate keys and functional dependencies must be identified correctly. This information is then used to remove partial and transitive dependencies.

Q8. Retail Inventory — Complete Normalization

Given Relation

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Functional Dependencies

Assume:

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Candidate Key

PID

Because PID uniquely identifies a product.

Transitive Dependency

We have:

PID → SupplierID

and:

SupplierID → SupplierName

Therefore:

PID → SupplierName

This is a transitive dependency.

Decomposition

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: PID

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

Primary Key: SupplierID

If a Product Can Be Stored in Multiple Warehouses

Then the warehouse relationship should be separated:

PRODUCT_WAREHOUSE

PRODUCT_WAREHOUSE(PID, Warehouse)

Primary Key:

(PID, Warehouse)

Final Design

- PRODUCT
- SUPPLIER
- PRODUCT_WAREHOUSE

Conclusion

The decomposition removes transitive dependencies and avoids unnecessary repetition of supplier information.

Under the stated assumptions, the resulting design can satisfy **3NF/BCNF**.

Q9. School ERP — Normalize up to BCNF

For this case, we can start with a poorly designed relation.

Unnormalized Relation

SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName)

Functional Dependencies

- StudentID → StudentName, ClassID
- ClassID → ClassName, TeacherID
- TeacherID → TeacherName
- SubjectID → SubjectName

Candidate Key

If one student can take multiple subjects:

(StudentID, SubjectID)

is the candidate key.

Partial Dependencies

StudentID → StudentName, ClassID

StudentID is only part of the composite key.

Also:

SubjectID → SubjectName

SubjectID is only part of the composite key.

Therefore, the relation violates **2NF**.

Transitive Dependencies

StudentID → ClassID

and:

ClassID → ClassName, TeacherID

Therefore:

StudentID → ClassName, TeacherID

Also:

TeacherID → TeacherName

Therefore:

StudentID → TeacherName

These are transitive dependencies.

Decomposition

STUDENT

STUDENT(StudentID, StudentName, ClassID)

CLASS

CLASS(ClassID, ClassName, TeacherID)

TEACHER

TEACHER(TeacherID, TeacherName)

SUBJECT

SUBJECT(SubjectID, SubjectName)

STUDENT_SUBJECT

STUDENT_SUBJECT(StudentID, SubjectID)

BCNF Check

In STUDENT:

StudentID → StudentName, ClassID

StudentID is the key.

In CLASS:

ClassID → ClassName, TeacherID

ClassID is the key.

In TEACHER:

TeacherID → TeacherName

TeacherID is the key.

In SUBJECT:

SubjectID → SubjectName

SubjectID is the key.

Therefore, each determinant is a superkey in its relation.

Conclusion

The School ERP schema is decomposed into smaller relations and can satisfy **BCNF** under the stated functional dependencies.

Q10. Movie Streaming Platform — 5NF and Lossless Join

Example Relation

Consider:

STREAM(Movie, Actor, Platform)

Suppose:

- A movie can have several actors.
- A movie can be available on several platforms.
- An actor can work with several platforms independently.

Example

Movie	Actor	Platform
M1	A1	P1
M1	A1	P2
M1	A2	P1
M1	A2	P2

This may create unnecessary combinations when the relationships are independent.

Join Dependency

The relation can be decomposed into:

MOVIE_ACTOR(Movie, Actor)

MOVIE_PLATFORM(Movie, Platform)

ACTOR_PLATFORM(Actor, Platform)

The original relation can be reconstructed by joining these relations if the appropriate join dependency holds.

Decomposition

MOVIE_ACTOR

Movie	Actor
M1	A1
M1	A2

MOVIE_PLATFORM

Movie	Platform
M1	P1
M1	P2

ACTOR_PLATFORM

Actor	Platform
A1	P1
A1	P2
A2	P1
A2	P2

5NF

A relation is in **5NF/PJNF** when every non-trivial join dependency is implied by the candidate keys.

The decomposition separates the independent relationships and prevents unnecessary redundancy.

Lossless-Join Verification

To verify lossless join:

1. Start with the original relation.
2. Project it onto the decomposed relations.
3. Join the projected relations.
4. Compare the reconstructed relation with the original relation.
5. If no spurious tuples are generated and the original tuples can be recovered, the decomposition is lossless.

Symbolically:

R = MOVIE_ACTOR ⋈ MOVIE_PLATFORM ⋈ ACTOR_PLATFORM

If the join produces exactly the original relation, the decomposition is **lossless**.

Conclusion

The movie streaming relation can be decomposed into smaller relations based on the join dependencies. If the join dependency is valid and reconstruction produces exactly the original relation, the decomposition is **lossless and in 5NF**.